SONOMA STATE UNIVERSITY

Hardware Implementation of Four Degree of Freedom Deep Reinforcement Learning Arm Control Model Trained Through Human Demonstrations

by

Roel J. Hernandez C.

A project submitted to

Sonoma State University

in partial fulfillment of the requirements

for the degree of

Master of Science

in

Electric and Computer Engineering

Supervisor: Dr. Sudhir Shrestha

Date:

Accepted by the Graduate School

_____ , _____

Date Dean of the Graduate School

The undersigned have examined the project entitled "Hardware Implementation of Four Degree of Freedom Deep Reinforcement Learning Arm Control Model Trained Through Human Demonstrations" presented by Roel Jose Hernandez Chaudary a candidate for the degree of Master of Science in Electric and Computer Engineering and hereby certify that it is worthy of acceptance.

_____

Date

_____

Date

_____

Date

DECLARATION OF AUTHORSHIP

I, Roel Jose Hernandez Chaudary declare that this project "Hardware Implementation of Four Degree of Freedom Deep Reinforcement Learning Arm Control Model Trained Through Human Demonstrations", **title of the work** and the work presented in it is my own. I confirm that:

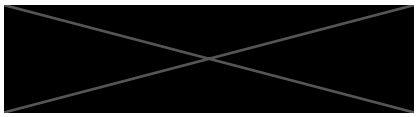
- This work was done wholly or mainly while in candidature for Master's degree at Sonoma State University.
- Where any part of this project has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this project is entirely my own work.
- I have acknowledged all main sources of help.
- Where the project is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

AUTHORIZATION FOR REPRODUCTION OF MASTER'S PROJECT

I grant permission for the print or digital reproduction of this project in its entirety, without further authorization from me, on the condition that the person or agency requesting reproduction absorb the cost and provide proper acknowledgment of authorship.

Date: Apr 22, 2022 Signature:

Street Address



City, State, Zip

Dedication

I dedicate this to every student and teacher that continues to pursue learning in this time of adversity. Also to my family, my parent for all their love, wisdom and support. Special thanks to my sister Hilda and my brother-in-law Kelly whom supported me to accomplish my Master.

Contents

Chapter 1

Introduction

Deep learning has provided new ways of manipulating, processing, and analyzing data. It sometimes may achieve results comparable to, or surpassing human expert performance, and has become a source of inspiration in the era of artificial intelligence. Another subfield of machine learning named reinforcement learning, tries to find an optimal behavior strategy through interactions with the environment. [1]

Robots were originally designed to assist or replace humans by performing repetitive and/or dangerous tasks which humans usually prefer not to do or are unable to do because of physical limitations imposed by extreme environments. There are different types of robots available, which can be grouped into several categories depending on their movement, Degrees of Freedom (DoF), and function. Articulated robots are among the most common robots used today. They look like a human arm and that is why they are also called robotic arm or manipulator arm [1] . In [2] a simulation of a two joint robot arm was trained to minimize the distance between the effector and the goal. The algorithms used was Deep Deterministic Policy Gradients (DDPG) that work well for robotic control problems. The arm learns reasonable. Pointing in direction of the goal which shows us that the reward function is robust to for the simulation. In [3] the simulation was made using OpenAi's Robotics Environment. The algorithm used was Deep Deterministic Policy Gradients (DDPG) with Hindsight Experience Replay (HER), is a new reinforcement learning algorithm that can learn from failure. HER can also learn successful policies from only sparse rewards. After doing some playing around and tested the HER+DDPG and DDPG. The author found that HER+DDPG was a lot faster at achieving the goal, with an epoch=200 the success was 94.6%.

In [4] the goal is trained a model artificial neural network (ANN) based adaptive nonlinear control approach of a robot arm, with highlight on its capability as a compliant control scheme. A 2DOF robot was used

in the study. The neural network consists of 3 layers. The network successfully learned after $t = 0.5s$ where the weights converge. The main benefit of using this algorithm is the simple and intuitive way of decoupling the adaptive ANN based control and compliant behavior making the controller design more intuitive. The algorithm was successfully applied to a simulated 2DOF robot arm for robust, adaptive, compliant behavior generation of the arm.

In [5] they present a few case studies of applications of deep Reinforcement Learning to various robotic tasks that we have studied. The applications span manipulation, grasping, and legged locomotion. The sensory inputs used range from low-dimensional proprioceptive state information to high-dimensional camera pixels, and the action spaces include both continuous and discrete actions. These case studies illustrate that deep Reinforcement Learning can, in fact, be used to learn directly in the real world, can learn to utilize raw sensory modalities such as camera images, and can learn tasks that present a substantial physical challenge, such as walking and dexterous manipulation. Most importantly, these case studies illustrate that policies trained with deep Reinforcement Learning can generalize effectively, such as in the case of the robotic grasping experiments.

In [6] The robot used in this implementation is a two-link arm with two rotational degrees of freedom restricted to movement in a plane. Two neural networks were used: the plant identification one and the other realize the controller function. To identify the plant, a Levenberg- Marquandt (trainlm) technique is used. Performance behavior of the identification plant process is measured through the mean square error (MSE) the NN controller based on reference model has been achieved. From the simulation results, the neural networks reproduce a model output from a plant.

In [7] a detailed and extensive comparison of the Trust Region Policy Optimization and Deep Q-Network with Normalized Advantage Functions with respect to other state of the art algorithms, namely Deep Deterministic Policy Gradient and Vanilla Policy Gradient was proposed. Both simulated and real-world experiments are provided. Deep Reinforcement Learning algorithms provides nowadays very general methods with little tuning requirements, enabling the robot learning of complex robotic tasks with deep neural networks. Such algorithms showed great potential in synthesizing neural nets capable of performing the learned task while being robust to physical parameters and environment changes. In [8] a simulated robot arm was tasked with generating trajectories. A non-expert human was used for feedback by defining goals, comparing, and selecting amongst possible trajectories. The researchers showed the preference elicitation in reinforcement learning were successful in learning unknown reward functions. In [9] the Deep Reinforcement Learning concentrates on the integration of neuromotor-control-inspired costs in DRL algorithms for robotic control. The results show that the learned policies, especially for model-free agents, can be significantly improved from

a real-world application point of view, by integrating modular costs, with performance approaching human reference scores on some metrics.

There are other works related to Deep Reinforcement learning and computing vision, such in [10] introduces a machine learning based system for controlling a robotic manipulator with visual perception only. In [11] uses 3D simulations to train a 7-DOF robotic arm in a control task without any prior knowledge. In [12] introduce QT-Opt, a scalable self-supervised vision-based reinforcement learning framework. In [13] demonstrate that a recent deep reinforcement learning algorithm based on off-policy training of deep Q-functions can scale to complex 3D manipulation tasks

Chapter 2

Theoretical Backgrounds

2.1 Reinforcement Learning

Reinforcement learning is the training of machine learning models to make a sequence of decisions. The agent learns to achieve a goal in an uncertain, potentially complex environment. In reinforcement learning, an artificial intelligence faces a game-like situation. The computer employs trial and error to come up with a solution to the problem. To get the machine to do what the programmer wants, the artificial intelligence gets either rewards or penalties for the actions it performs. Its goal is to maximize the total reward [14]. The learning system, called an agent in this context, can observe the environment, select, and perform actions, and get rewards in return or penalties in the form of negative rewards. It must then learn by itself what is the best strategy, called a policy, to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation [15]. The environment is typically stated in the form of a Markov decision process (MDP) because many reinforcement learning algorithms for this context use dynamic programming techniques [16]. Our robot arm is our agent's environment. States are the positions in our environment that our agent can reach. The different permutations of joint positions are our states. The agent reaches different states by performing actions [17]. Basic reinforcement is modeled as a Markov decision process (MDP):

- A set of environment and agent states, S.
- A set of actions, A, of the agent.
- $Pa(s, s') = Pr(st + 1 = s'|st = s, at = a)$ is the probability of transition (at time t) from state s to state s' under action a.

- $Ra(s, s')$ is the immediate reward after transition from s to s' with action a.

2.2 Q-Learning

Q-Learning is a Reinforcement learning policy that will find the next best action, given a current state. It chooses this action at random and aims to maximize the reward.

Important Terms in Q-Learning

- States: The State, S, represents the current position of an agent in an environment.
- Action: The Action, A, is the step taken by the agent when it is in a particular state.
- Rewards: For every action, the agent will get a positive or negative reward.
- Episodes: When an agent ends up in a terminating state and can't take a new action.
- Q-Values: Used to determine how good an Action, A, taken at a particular state, S, is $Q(A, S)$.
- Temporal Difference: A formula used to find the Q-Value by using the value of current state and action and previous state and action [18].

Using the reward feedback, the agent assigns values to each state action pair determined by Equation 2.1.

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma max_a Q(s', a')]. \tag{2.1}$$

Q-Learning poses an idea of assessing the quality of an action that is taken to move to a state rather than determining the possible value of the state (value footprint) being moved to [19].

2.3 Robot Arm

Robot Manipulators are composed of links connected by joints to form a kinematic chain. Joints are typically rotary (revolute) or linear (prismatic). A revolute joint is like a hinge and allows relative rotation between two links. A prismatic joint allows a linear relative motion between two links. Each joint represents the interconnection between two links. We denote the axis of rotation of a revolute joint, or the axis along which a prismatic joint translates by z_i if the joint is the interconnection of links i and $i + 1$. The joint variables, denoted by θ for a revolute joint and d for the prismatic joint, represent the relative displacement between

adjacent links. A configuration of a manipulator is a complete specification of the location of every point on the manipulator. The set of all possible configurations is called the configuration space. An object is said to have n degrees-of-freedom (DOF) if its configuration can be minimally specified by n parameters. Thus, the number of DOF is equal to the dimension of the configuration space. For a robot manipulator, the number of joints determines the number DOF. The workspace of a manipulator is the total volume swept out by the end effector as the manipulator executes all possible motions. The workspace is constrained by the geometry of the manipulator as well as mechanical constraints on the joints [20].

2.4 Denavit-Hartenberg Parameters

The Denavit-Hartenberg (DH) convention defines a configuration for our robot arm. In this convention, each homogeneous transformation A_i is represented as a product of four basic transformations where the four quantities θ_i, a_i, d_i, α_i are parameters associated with link i and joint i.

$$A_i = Rot_{z,\theta_i} Trans_{z,d_i} Trans_{x,a_i} Trans_{x,\alpha_i} \tag{2.2}$$

$$= \begin{bmatrix} C_{\theta_i} & -S_{\theta_i} & 0 & 0 \\ S_{\theta_i} & C_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & C_{\alpha_i} & -S_{\alpha_i} & 0 \\ 0 & S_{\alpha_i} & C_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} C_{\theta_i} & -S_{\theta_i} C_{\alpha_i} & S_{\theta_i} S_{\alpha_i} & a_i C_{\theta_i} \\ S_{\theta_i} & C_{\theta_i} C_{\alpha_i} & -C_{\theta_i} S_{\alpha_i} & a_i S_{\theta_i} \\ 0 & S_{\alpha_i} & C_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The four parameters a_i, α_i, d_i, and θ_i in 2.2 are generally given the names link length, link twist, link offset, and joint angle, respectively.

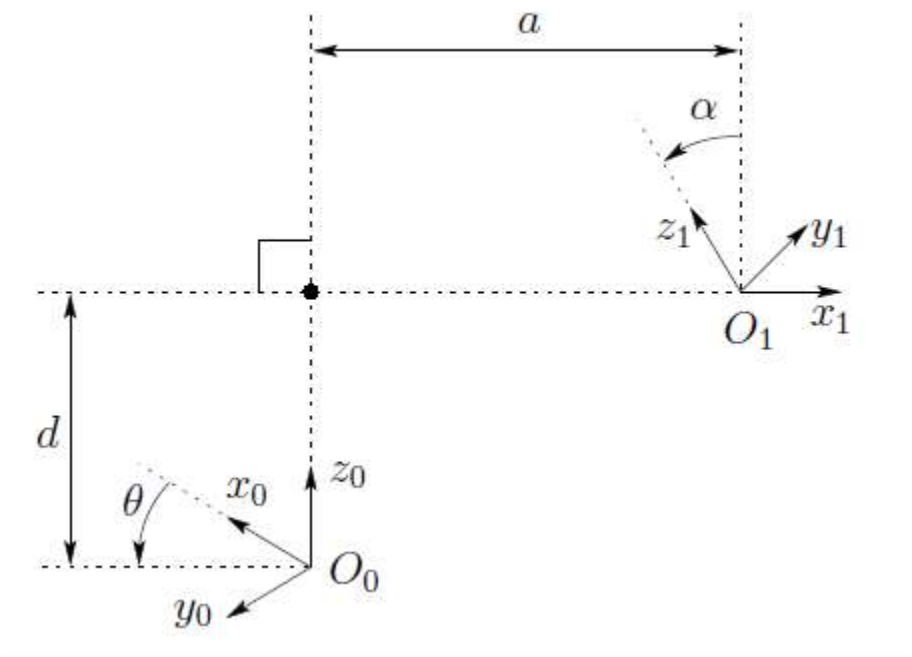


Figure 2.1: Denavit-Hartenberg convention between two joints

2.5 Neural Network

A neural network is a series of algorithms that endeavors to recognize underlying relationships in a set of data through a process that mimics the way the human brain operates. In this sense, neural networks refer to systems of neurons, either organic or artificial in nature. Neural networks can adapt to changing input; so, the network generates the best possible result without needing to redesign the output criteria [21].

Artificial neural networks (ANNs) are comprised of a node layer, containing an input layer, one or more hidden layers, and an output layer. Each node, or artificial neuron, connects to another and has an associated weight and threshold. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network [22].

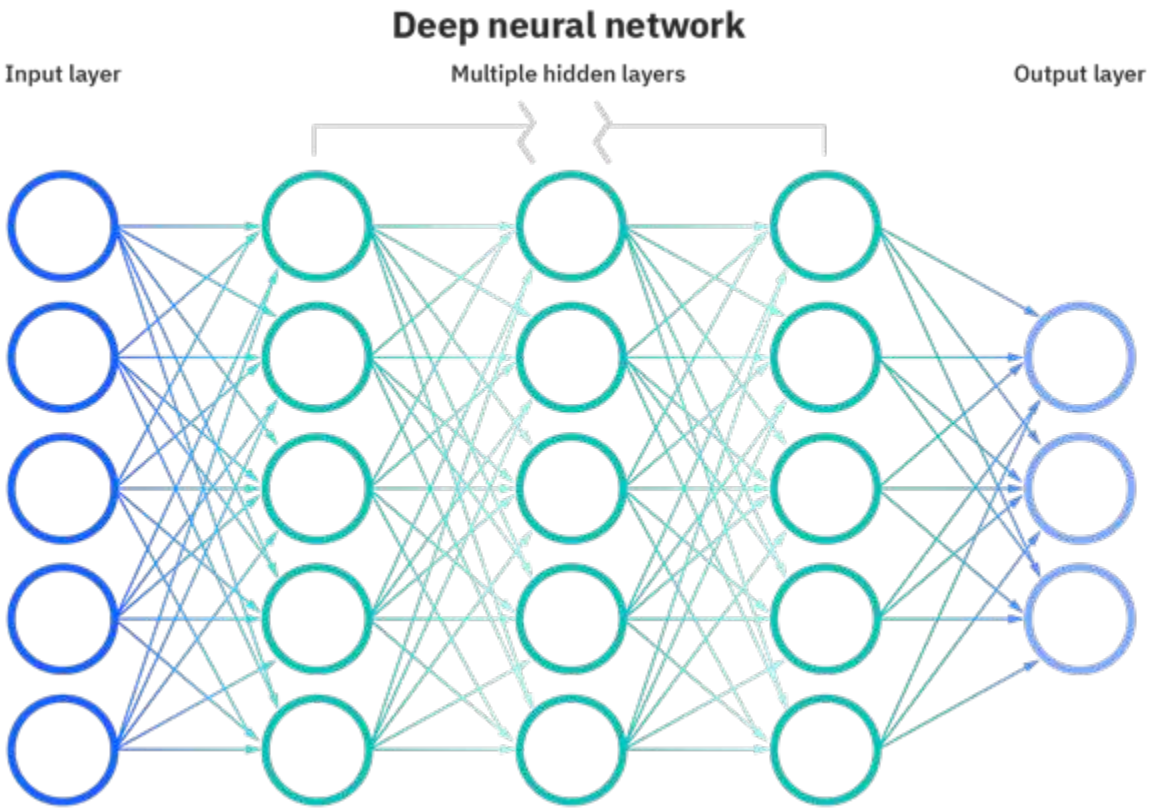


Figure 2.2: Neural Network

Chapter 3

Methodology

3.1 Implementation

In this project I will implement in Raspberry Pi a neural model trained with "deep reinforcement learning for a four degree of freedom robot arm control simulation accelerated by human demonstrations" to control the robotic arm with python [17]. The Robot Arm is a Dobot Magician Robot Arm. Each joint is a revolute joint creating an R,R,R,R - waist, shoulder, elbow, and wrist configuration [23]. In polar coordinates, the waist axis controls the θ position and the remaining 3 axes control the radius r and height h. The limits of each axis are: $\theta_1 = 180^o$, $\theta_2 = 85^o$, $\theta_3 = 100^o$, $\theta_4 = 90^o$ [23].

Neural Network Configuration

- Input layer size = 7
- x, y, z, $\theta_1, \theta_2, \theta_3, \theta_4$
- Output = 81, state value
- 81 Q value prediction
- 5 hidden layers 128 nodes, greater than Q values
- Activation function: ELU Exponential Linear Unit
- Adam optimizer

- Huber loss function
- Learning rate 0.001
- Batch size 64

The following table summarize the Python program modules used to run the simulation

Module	**Description**
test_trained.py	main module
class_DQNAgent_T.py	Python class to check the limits and perform the movement of the arm
class_QNetwork.py	Python class to define the Neural network model
class_ReplayMemory.py	Python class to store/recover example moves whether generated by random exploration or human demonstration
class_RobotArmEnvV2.py	Python class to perform the forward kinematics move calculation, generate random angles and position
robot_arm_controller2.py	Python class to communicate with the robotic Arm and perform the control of it.

Raspberry Pi Device used to run the trained model algorithm:

- Raspberry Pi 3B+
- OS: Raspbian GNU/linux 10 (buster)
- Python version: Python 3.7.3

To perform the trained model test, a random target position and start angle set are chosen to run the neural network algorithm. The test is carried out to verify the accuracy of the trained model and how efficient the training process was and if it needs to be optimized or needs more data to improve its efficiency. The test was performed by executing the trained algorithm 10 times with the same input parameters (target position and starting θ angles) to determine if in each execution of the algorithm the movement pattern is similar or if they differ greatly between each execution.

The destination position is calculated as follows:
a random value between -75 and 75 is chosen for p (initial angle of the 180 possible angles that θ angle can

have) and a random value for r between 50 and 90 (value that is within the robot's working space range). With these values, x and y coordinates are calculated in the following way $x = r * cos(p)$, $y = r * sin(p)$, z is always initialized as 0. The initial angles are obtained as follows: θ_1 value random between -75 and 75, θ_2 = 70, θ_3 = 75 and θ_4 random value between 25 and 75.

3.2 Metrics used

I used 5 different metrics to evaluate the accuracy of the model

Euclidean Distance:

The Euclidean distance between two points in space measures the length of a segment connecting the two points.

$$d(P_t, P_p) = \sqrt{(X_t - X_p)^2 + (Y_t - Y_p)^2 + (Z_t - Z_p)^2} \tag{3.1}$$

Distance Error Function:

When the target is 180 degrees away from the target position, the move in either direction reward the agent equally, so, we used an altered error function as the shortest arc length plus the euclidean between the radius and height variables [17].

$$error = r \times |\Delta\theta| + \sqrt{\Delta r^2 + \Delta h^2} \tag{3.2}$$

3.3 Hardware Implementation

Hardware and Software used to implement the trained model:

- Raspberry Pi 3B+
  - 1 GB LPDDR2 SDRA
  - Broadcom BCM2837B0, Cortex-A53 (ARMv8) 64-bit Soc @ 1.4GHz
  - OS: Raspbian GNU/linux 10 (buster)
- Robot:
  - Dobot Magician

- Software:
  - Python 3.7.3
  - TensorFlow 2.0

Chapter 4

Results and Discussion

4.1 Simulations Results

I run the simulation for the trained model 5,000 episodes (the same number of episodes used to train the model[17]) to evaluate the accuracy of the trained model. I ran the simulation with two different kind of 4 DOF Robotic Arm. One, the ArbotiX Robot Arm used during the model trained phase [17], the other one, Dobot Magician Robot Arm.

Table 4.1 shows some statistics of the simulations performed with both robot. For example, we can observe that the mean value of the distance error and the Euclidean distance are within the range of acceptance, 3.051 cm and 3.520 cm respectively for Arbotix Robot Arm. The distance error and the Euclidean distance are 6.155 cm and 5.723 cm respectively for Dobot Magician Robot Arm. Calculating the standard deviations shows that for the distance error there is a dispersion of 2.632 cm with respect to the mean and for the Euclidean distance of 1.050 cm, which indicates that there is a good behavior of the simulation in reaching or approaching the target point for the Arbotix Robot Arm. For the Dobot Magician Robot Arm the standard deviations shows that for the distance error there is a dispersion of 1.759 cm with respect to the mean and for the Euclidean distance of 1.595 cm. The rmse value for Arbotix Robot Arm confirms that there is a good behavior of the simulation, since it is 2.121 cm. The rmse value for the Dobot magician Robot Arm is 3.430 cm.

Table 4.2 shows the number of episodes and percentages that fall within the acceptance regions. For example, for ArbotiX Robot Arm, we can observe that the percentage of episodes with distance error less than 1 cm is 18.32% of the simulations and less than 5 cm is 83.96%, which again shows a good behavior of the robot. Regarding the distance error and reward we can see that in this case for distance error < 5 cm and reward <200

Table 4.1: Simulation Statistics Results

Statistic	**Value**	
	ArbotiX	**Dobot**
Average Distance Error	3.051 cm	6.155 cm
Average Euclidean Distance	3.520 cm	5.73 cm
Average Reward	11.797	1.653
Distance Error Variance	69.274 cm	30.939 cm
Euclidean Distance Variance	11.031 cm	25.441 cm
Reward Variance	818.484	92.941
Distance Error Standard Deviation	2.632 cm	1.759 cm
Euclidean Distance Standard Deviation	1.050 cm	1.595 cm
Reward Standard Deviation	9.047	3.049
RMSE	2.121 cm	3.430 cm

there are 3,085 episodes, which represents 61.70% of all simulations lying within the acceptance margins, so we can mention that the behavior of the simulation is satisfactory for this Robot. For Dobot Magician Robot arm, we can observe that there is not of episodes with distance error less than 1 cm and with distance error less than 5 cm is 27.48%, which is a low percentage this Robot. Regarding the distance error and reward we can see that in this case for distance error < 5 cm and reward <200 there are 1,374 episodes, which represents 27.48% of all simulations lying within the acceptance margins, so we can mention that the behavior of the simulation is not satisfactory for Dobot Magician Robot Arm.

In figure 4.1 show the comparative distance error base on equation 3.2 for both Robot arm. The Arbotix robot agent reached a minimum position error of 0.75 cm in 207 steps with a reward of 174. The Dobot Magician robot agent reached a minimum position error of 1.7 cm in 78 steps with a reward of 59.

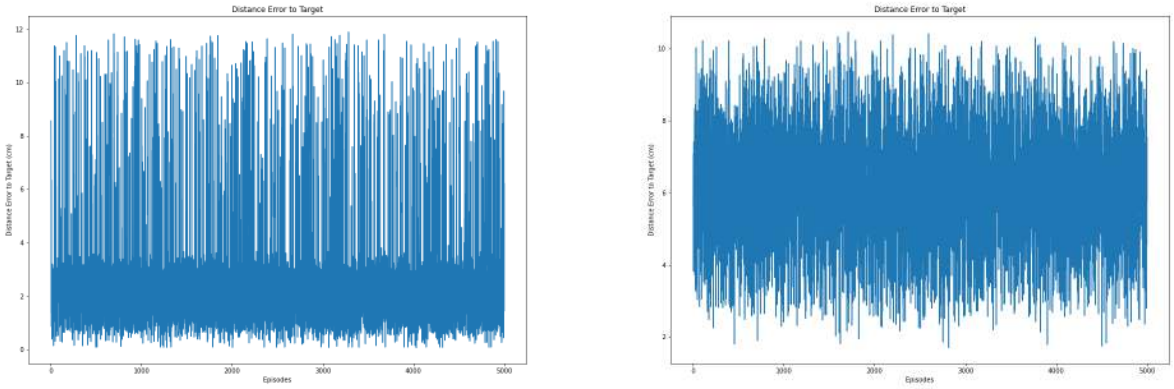
In figure 4.2 show the comparative Euclidean distance 3.1 for both Robot arm. The Arbotix robot agent reached a minimum euclidean distance of 0.45 cm in 81 steps with a reward of 60. The Dobot Magician robot agent reached a minimum euclidean distance of 1.7 cm in 815 steps with a reward of 66.

In figure 4.3 show the comparative Rewards obtained during the simulation for bot robot agent with the trained model. For ArbotiX Robot Arm we can see 128 episode with a minimum reward of -50 [1] and 148

[1]Stop condition of Algorithm "rewards < -50"

Table 4.2: Summary Simulation Results

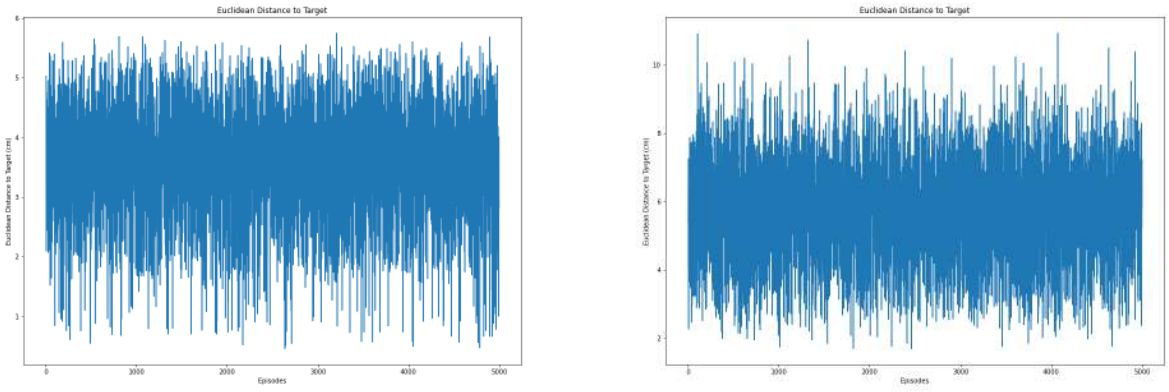
Condition	ArbotiX		Dobot	
	Value	Percents	Value	Percents
Numbers Episodes with Distance Error < 1 cm	916	18.32%	0	0.00%
Numbers Episodes with Distance Error < 5 cm	4,198	83.96%	1,374	27.48%
Distance Error<5 cm and Rewards<100	1,505	30.10%	1,350	27.00%
Distance Error<5 cm and Rewards<200	3,085	61.70%	1,374	27.48%
Distance Error<1 cm and Rewards<100	203	4.06%	0	0.00%
Distance Error<1 cm and Rewards<200	635	12.70%	0	0.00%



(a) ArboitX.

(b) Dobot.

Figure 4.1: Distance error for a 4-DOF robot agent simulation results for 5,000 episodes at 300 steps with human trained model.



(a) ArboitX.

(b) Dobot.

Figure 4.2: Euclidean Distance for a 4-DOF robot agent simulation results for 5000 episodes at 300 steps with human trained model.

episodes with the maximum reward of 300 [2]. For the Dobot Magician Robot Arm we can see 1,311 episodes with a reward of -10 [3] and 2 episodes with a reward of 152, none of the episodes for Dobot Magician Robot Arm reached the maximum reward of 300.

4.2 Robot Arm Results

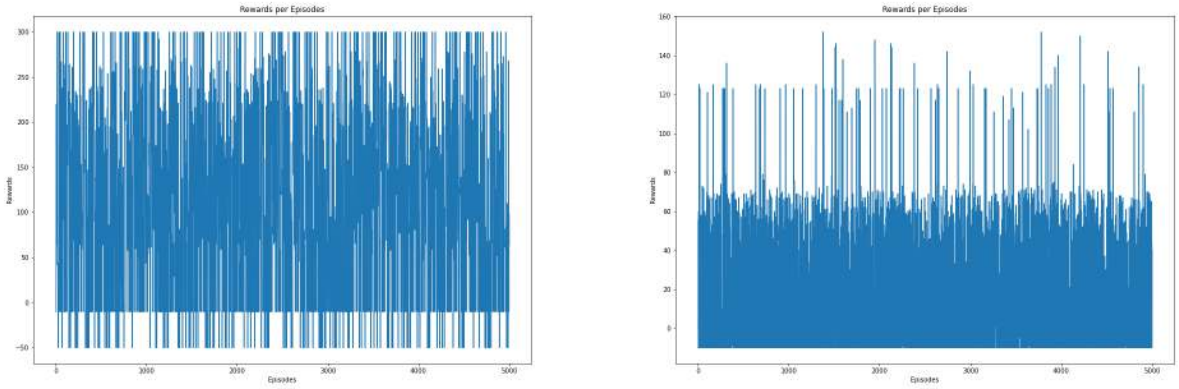
I run the trained model 50 times using the Dobot Magician Robotic arm with the following results.

Table 4.3 shows some statistics of the execution performed with the robot Dobot magician. For example, we can observe that the mean value of the distance error and the Euclidean distance are within the range of 6.796 cm and 6.164 cm respectively. Calculating the standard deviations shows that for the distance error there is a dispersion of 1.323 cm with respect to the mean and for the Euclidean distance of 1.312 cm, which indicates that there is room for improvement.

Table 4.4 shows the number of episodes and percentages that fall within the acceptance regions. For example, we can observe that the percentage of episodes with distance error less than 5 cm is only 10%.

[2]Stop condition of Algorithm "reward > 300"

[3]Stop condition of Algorithm, 10 consecutive negative rewards"



(a) ArboitX.

(b) Dobot.

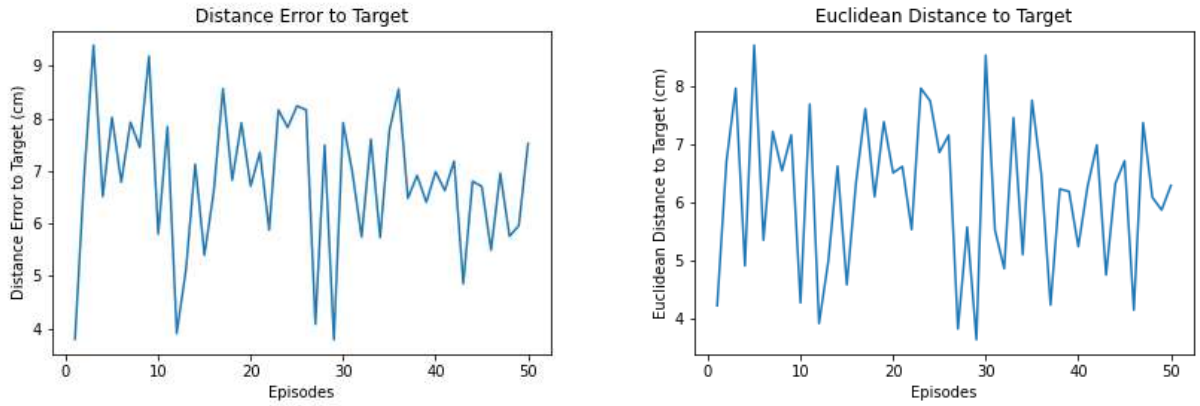
Figure 4.3: Rewards for a 4-DOF robot agent simulation results for 5000 episodes at 300 steps with human trained model.

Table 4.3: Dobot Magician Statistics Results

Statistic	**Value**
Average Distance Error	6.796 cm
Average Euclidean Distance	6.164 cm
Distance Error Variance	17.513 cm
Euclidean Distance Variance	17.201 cm
Distance Error Standard Deviation	1.323 cm
Euclidean Distance Standard Deviation	1.312 cm
RMSE	3.637 cm

Table 4.4: Dobot Magician Statistics Results

Condition	Value	Percents
Numbers Episodes with Distance Error < 1 cm	0	0.00%
Numbers Episodes with Distance Error < 5 cm	5	10.00%
Distance Error<5 cm & Step<100	5	10.00%
Distance Error<5 cm & Step<200	5	10.00%
Distance Error<1 cm & Step<100	0	0.00%
Distance Error<1 cm & Step<200	0	0.00%



(a) Distance Error.

(b) Euclidean Distance.

Figure 4.4: Distance error and Euclidean Distance for a 4-DOF robot agent dobot results for 50 episodes at 300 steps with human trained model.

In figure 4.4 show the distance error base on equation 3.2 and Euclidean distance 3.1, the Robot Arm reached a minimum position error of 3.7 cm in 47 steps. The minimum euclidean distance reached was 3.6 cm in 47 steps.

4.3 Discussion

From the simulation result between the two Robot Arm and the execution performed with the robot Dobot magician we can see a better performance with the Arbotix Robot arm. The difference between the simulation

and the implementation in the robot are due to working space in each robot are different. The Arbotix Robot has limits of each axis: $\theta_1 = 300^o$, $\theta_2 = 180^o$, $\theta_3 = 200^o$, $\theta_4 = 210^o$, while the Dobot magician has limits of each axis: $\theta_1 = 180^o$, $\theta_2 = 85^o$, $\theta_3 = 100^o$, $\theta_4 = 90^o$. Also, the model was trained with the parameter for ArbotiX Robot Arm and its Human Demonstration dataset.

Other works done before has similar results as we get in our project, i.e. [11] using G-learning and running 50 episodes with $\epsilon = 0.1$ has a successful rate of 64%, while [5] With QT-Opt Algorithm with 5,000 real-world data reaches a success rate of 91%. Similar to the rate reaches by the Simulation with ArbotiX Robot agent of 84%.

The Dobot Magician Robot Agent reached a successful rate of 27%. To increase the accuracy and overcome the challenges of training robot with deep reinforcement learning we can use generalization, specifically data diversity as state in [5] "Good data diversity that covers the space of generalization we care about is critical to have good performance with deep learning." We can train the model using a dataset that combined the human demonstration position for different Robot Arm and then implement the trained model with each robot to compare the behavior.

Chapter 5

Conclusion and Future work

5.1 Conclusion

We were able to implement the trained model on hardware (Raspberry Pi) and control the robotic arm via the interface in Python. The robotic arm moved smoothly closer to the target position using the model trained through human demonstration. The model struggled close to target because of our errors in forward kinematic calculation. When deployed on a physical robot arm and combined with an object detection system, our robot agent could be utilized in a peg insertion application or pick and place.

5.2 Future work

5.2.1 Robot Arm

We can implement our trained model in different robot arm making the adjustment in the DH parameters to compare the accuracy to reach the target position. The accuracy will be better with Robot Arm that has similar characteristic to ArbotiX Robot Arm. If we train the model with a bigger set of data we can have a good behavior and accuracy in the implementation.

5.2.2 Natural Language Processing

The agent learned to select actions based upon deep Q learning from example steps. However, it would be a great idea to add natural language processing to the action selection since the technology is already available

to process voice. This would allow a human to control the robot arm via English instruction set.

5.2.3 Microcontroller and Arduino

We can implement the trained model in 32-bit microcontroller using Arduino to control the robotic arm.

Appendix A

Robot Arm Specification

Figure A.1: Dobot Magician

Robot Arm Specifications [25]

- Name: Dobot Magician
- Maximum payload: 500g
- Maximum reach: 320mm
- Motion range:
  - Base: $-90^o \sim +90^o$
  - Rear Arm: $0^o \sim 85^o$
  - Forearm: $-10^o \sim +90^o$
  - End-effector rotation: $-90^o \sim +90^o$
- Maximun speed (with 250g payload):
  - Rotational speed of Rear arm, Forearm and base: $320^o/s$
  - Rotational speed of servo: $480^o/s$
- Repeated positioning accuracy: 0.2mm
- Power supply: 100V-240V AC, 50/60Hz
- Power in: 12V/7A DC
- Communication: USB, WIFI, Bluetooth
- I/O: 20 extensible I/O interfaces
- Working temperature: $-10^oC \sim 60^oC$

A.1 Appearance and Constitute

Dobot Magician consists of a Base, Rear Arm, Forearm, and end-effector, etc. Figure A.2 shows the appearance.

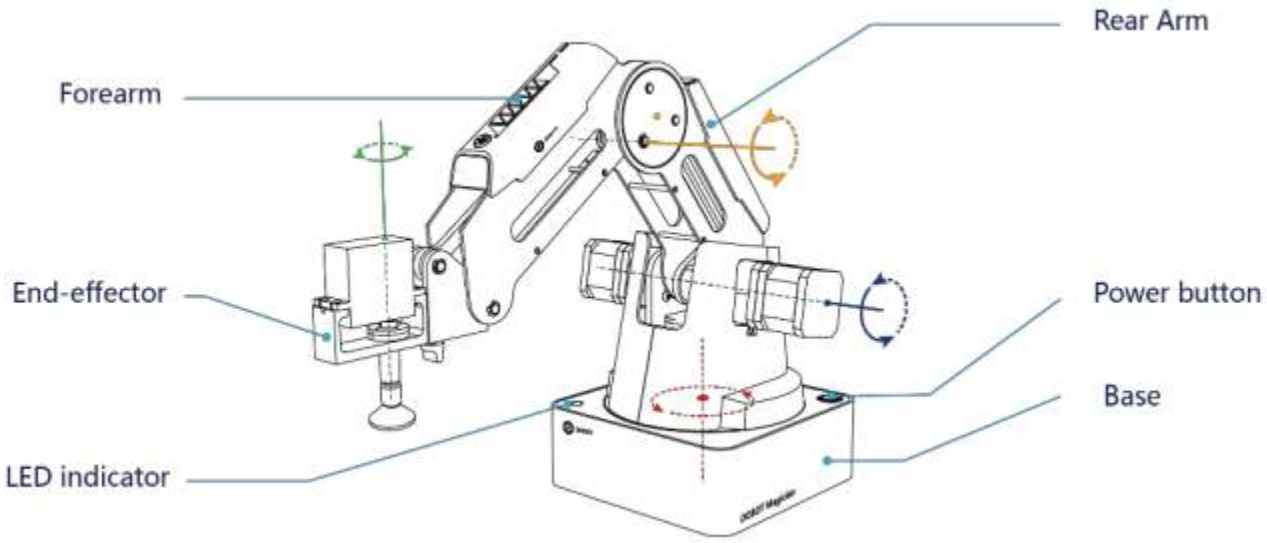


Figure A.2: The appearance of Dobot Magician

A.2 Working Principle

This section describes the workspace, principle, size and technical specifications of Dobot Magician.

A.2.1 Workspace

Figure A.3 and Figure A.4 shows the workspace.

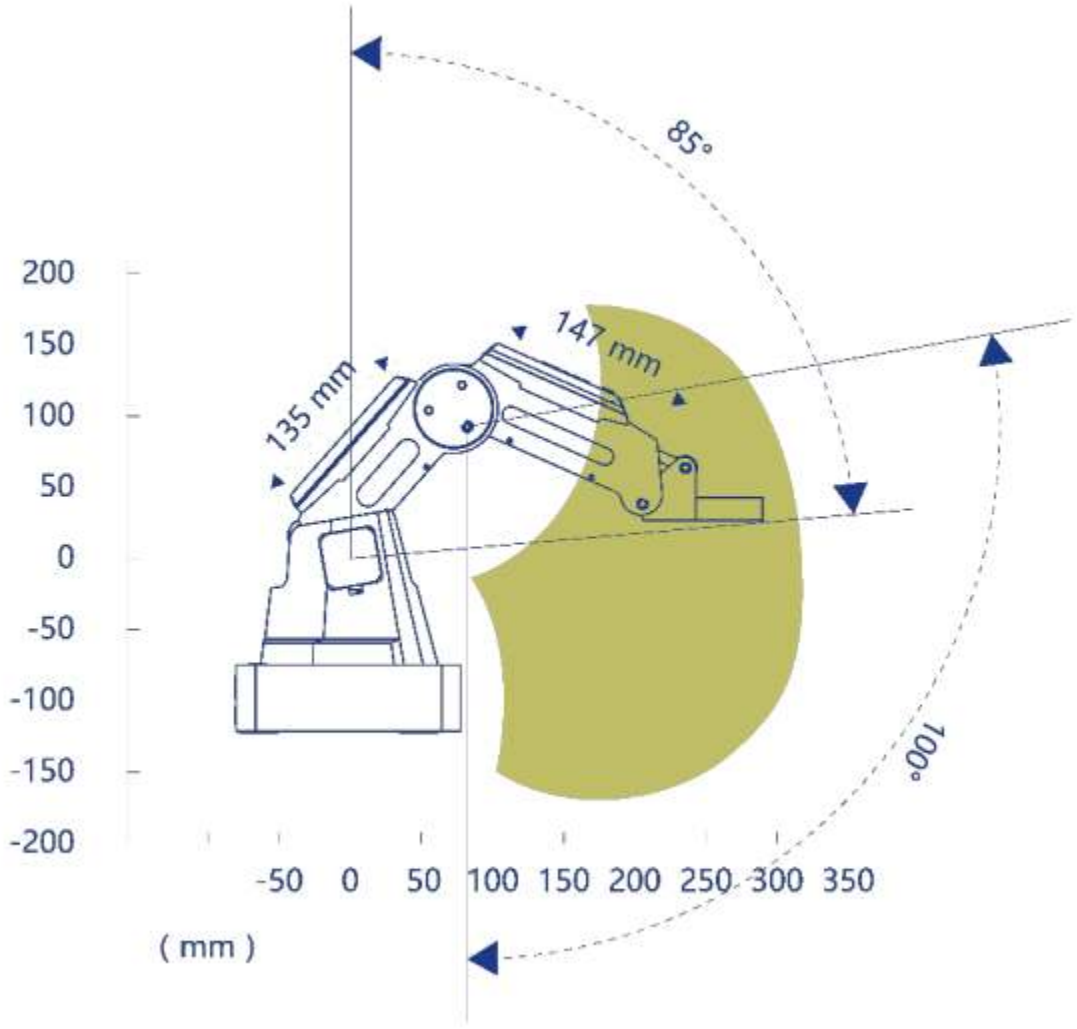


Figure A.3: Workspace of Dobot Magician (1)

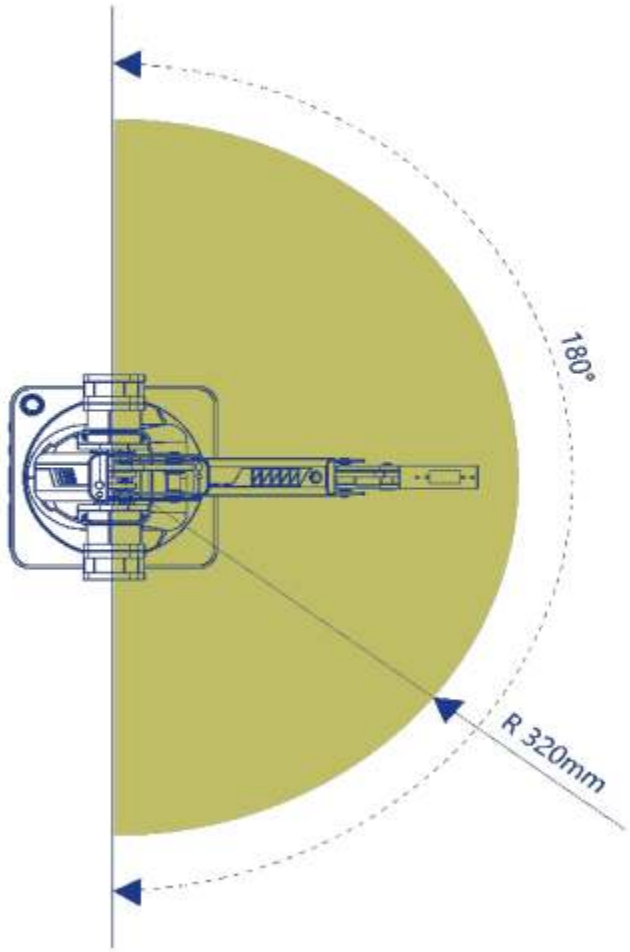


Figure A.4: Workspace of Dobot Magician (2)

A.2.2 Coordinate System

Dobot Magician has two types of coordinate system, the joint one and the Cartesian one, as shown in Figure A.5 and Figure A.6 respectively.

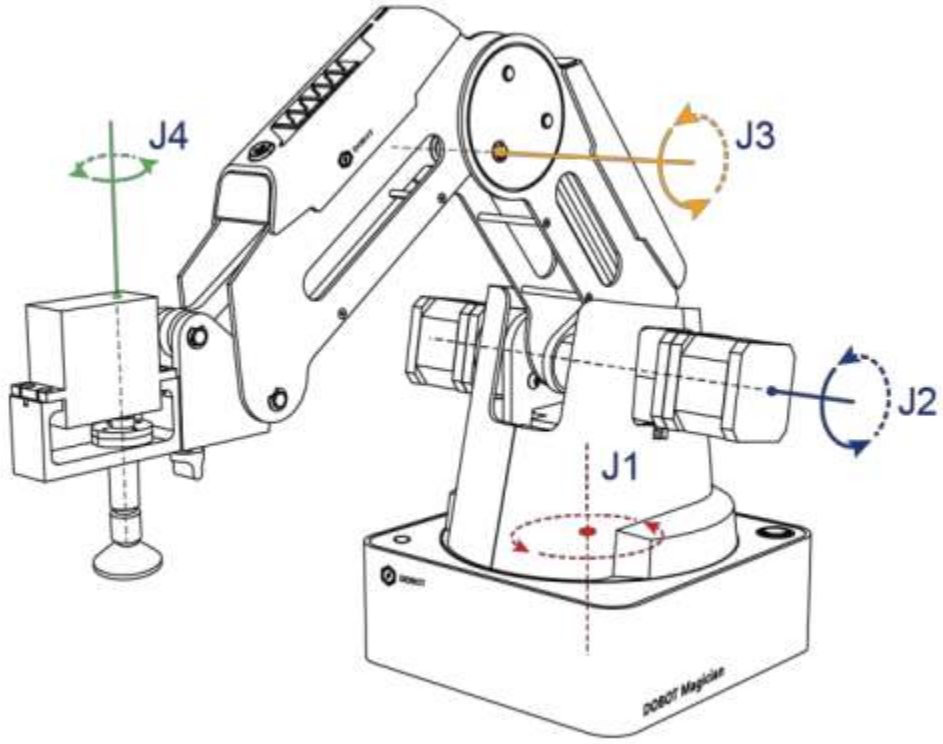


Figure A.5: Joint coordinate system

- Joint coordinate system: The coordinates are determined by the motion joints.
- If the end-effector is not installed, Dobot Magician contains three joints: J1, J2, and J3, which are all the rotating joints. The positive direction of these joints is counter-clockwise.
- If the end-effector with servo is installed, such as suction cup kit, gripper kit, Dobot Magician contains four joints: J1, J2, J3 and J4, which are all the rotating joints. The positive direction of these joints is counter-clockwise.
- Cartesian coordinate system: The coordinates are determined by the base.
- The origin is the center of the three motors (Rear Arm, Forearm, base).

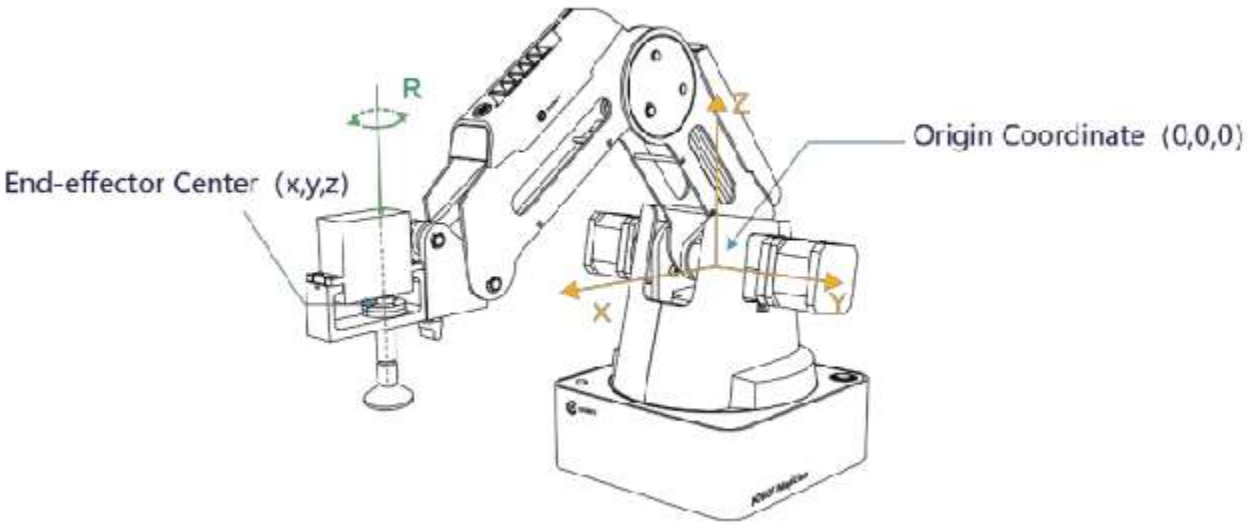


Figure A.6: Cartesian coordinate system

- The direction of X-axis is perpendicular to the base forward. The direction of Y-axis is perpendicular to the base leftward.
- The direction of Z-axis is vertical upward, which is based on the right hand rule.
- The R-axis is the attitude of the servo center relative to the origin of the robotic arm, of which the positive direction is counter-clockwise. The R-axis only exists once the end-effector with servo is installed.

Bibliography

[1] RONGRONG, LIU; FLORENT, NAGEOTTE; PHILIPPE, ZANNE; MATHELIN, MICHEL DE; DRESP-LANGLEY, BIRGITTA, "Deep Reinforcement Learning for the Control of Robotic Manipulation: A Focussed Mini-Review", 24 January 2021. [Online]. Available: https://doi.org/10.3390/robotics10010022

[2] SAROUFIM, MARK, "Controlling a 2D Robotic Arm with Deep Reinforcement Learning", 14 February 2019. [Online]. Available: https://blog.floydhub.com/robotic-arm-control-deep-reinforcement-learning/

[3] IMRAN, ALISHBA, "Training a Robotic Arm to do Human-Like Tasks using RL", 28 February 2019. [Online]. Available: https://medium.datadriveninvestor.com/training-a-robotic-arm-to-do-human-like-tasks-using-rl-8d3106c87aaf

[4] VINCE JANKOVICS, STEFAN MÁTÉFI-TEMPFLI AND PORAMATE MANOONPONG, "Artificial neural network based compliant control for robot arms."

[5] JULIAN IBARZ, JIE TAN, CHELSEA FINN, MRINAL KALAKRISHNAN, PETER PASTOR AND SERGEY LEVINE, "How to train your robot with deep reinforcement learning: lessons we have learned", The International Journal of Robotics Research, vol. 40, no. 4, p. 698-721, 2021.

[6] RAFID A. KHALIL, RAKAN KH. ANTAR, "Neural Network Based on Model Reference Using for Robot Arm Identification and Control", Al-Rafidain Engineering, vol. 22, no. 4, 2014.

[7] ANDREA FRANCESCHETTI, ELISA TOSELLO, NICOLA CASTAMAN, AND STEFANO GHIDONI, "Robotic Arm Control and Task Training through Deep Reinforcement Learning", 2020.

[8] P. CHRISTIANO, J. LEIKE, T. B. BROWN, M. MARTIC, S. LEGG, AND D. AMODEI, "Deep reinforcement learning from human preferences", arXiv: 1706.03741, 2017.

[9] MANON FLAGEAT, KAI ARULKUMARAN AND ANIL A. BHARATH, "Incorporating Human Priors into Deep", ISBN 978-2-87587-074-2., 2020.

[10] FANGYI ZHANG, JÜRGEN LEITNER, MICHAEL MILFORD, BEN UPCROFT, PETER CORKE, "Towards Vision-Based Deep Reinforcement Learning for Robotic Motion Control", 2015.arXiv:1511.03791v2, 2015.

[11] JOHNS, STEPHEN JAMES AND EDWARD, "3D Simulation for Robot Arm Control with Deep Q-Learning", arXiv:1609.03759v2, 2016.

[12] DMITRY KALASHNIKOV, ALEX IRPAN, PETER PASTOR, JULIAN IBARZ, ALEXANDER HERZOG, ERIC JANG, DEIRDRE QUILLEN, ETHAN HOLLY, MRINAL KALAKRISHNAN, VINCENT VANHOUCKE, SERGEY LEVINE, "QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation", arXiv:1806.10293v3, 2018.

[13] SHIXIANG GU, ETHAN HOLLY, TIMOTHY LILLICRAP AND SERGEY LEVINE, "Deep Reinforcement Learning for Robotic Manipulation with Asynchronous Off-Policy Updates", arXiv:1610.00633v2, 2016.

[14] BUDEK, OSIŃSKI AND KONRAD, "What is reinforcement learning? The complete guide", 5 July 2018. [Online]. Available: `https://deepsense.ai/what-is-reinforcement-learning-the-complete-guide/`

[15] GÉRON, AURÉLIEN, "Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow Concepts, Tools, and Techniques to Build Intelligent Systems", Oâ€™Reilly Media, ISBN: 978-1-492-03264-9, 2019.

[16] VAN OTTERLO, M.; WIERING, M., "Reinforcement learning and markov decision processes. Reinforcement Learning. Adaptation, Learning, and Optimization", ISBN 978-3-642-27644-6, 2012.

[17] DURAN, KIRK, "Deep reinforcement learning for a four degree of freedom robot arm control simulation accelerated by human demonstrations", Sonoma State University, 2021.

[18] SIMPLILEARN, "Machine Learning Tutorial: A Step-by-Step Guide for Beginners," [Online]. Available: `https://www.simplilearn.com/tutorials/machine-learning-tutorial`

[19] PAUL, SAYAK, "An introduction to Q-Learning: Reinforcement Learning," 15 May 2019. [Online]. Available: `https://blog.floydhub.com/an-introduction-to-q-learning-reinforcement-learning/`

[20] MARK W. SPONG, SETH HUTCHINSON, AND M. VIDYASAGAR, "Robot Modeling and Control", JOHN WILEY & SONS, INC.

[21] CHEN, JAME, "Neural Network", 22 December 2020. [Online]. Available: https://www.investopedia.com/terms/n/neuralnetwork.asp

[22] IBM CLOUD EDUCATION, "Neural Networks", 17 August 2020. [Online]. Available: https://www.ibm.com/cloud/learn/neural-networks

[23] "Phantomx reactor robot arm kit", [Online]. Available: https://www.trossenrobotics.com/p/phantomx-ax-12-reactor-robot-arm.aspx

[24] GROVER, PRINCE, "5 Regression Loss Functions All Machine Learners Should Know. Choosing the right loss function for fitting a model", 5 June 2018. [Online]. Available: https://heartbeat.comet.ml/5-regression-loss-functions-all-machine-learners-should-know-4fb140e9d4b0

[25] "Dobot Magician User Guide", Issue: V1.7.0, Date: 2019-01-09. Shenzhen Yuejiang Technology Co., Ltd